SpecMine: From Completion Annotation to Cutoff Datasets

Experiment report | CSC580, Group 7 | Work conducted 23–24 October 2026

1. Objective and current outcome

The longer-term objective is to predict the outcome of specification-driven work before that outcome occurs. This experiment first established a reviewable evidence base for completion, then paired the reviewed outcomes with features restricted to earlier observation windows. We produced consolidated source tables, an annotated cohort of 564 specification records, and cutoff datasets for days 7, 14, 30, 60 and 90. No predictive model has been trained. The cutoff tables are preparation for training; unresolved baseline states and missing negative outcomes still prevent a defensible supervised forecasting evaluation.

2. Source data and consolidation

We used the locally cloned SpecMine v1.1 GitHub sample, rather than the full Hugging Face mirror. The release contains 500 repositories across overlapping data layers. Our base uses the broad-census layer: 28,583 specification records from 415 repositories. Separate Kiro artifacts were excluded because their identifiers and document grain differ. The data snapshot is from July 2026; the reviewed records' specification histories were collected on July 11. Downloading these files later does not extend that historical coverage.

The new completion_stage1_prepare.py combines spec_files, spec_content_features and history summaries from spec_file_commits, supplemented by raw Markdown from specs.jsonl.gz. The record key is file_url_sha16, not the content-blob file_sha. Content metrics receive the feature_ prefix and experimental summaries receive exp_. Original columns and repeated repository/path records are preserved. The original feature_pipeline.py is unchanged.

Task associations come from spec_links and openspec_artifact_files; pull_requests, pr_files and openspec_code_refs supply additional review leads. One-to-many relationships remain separate to avoid multiplying specification rows. A shared task list or linked PR is evidence to investigate, not an automatic completion label.

Stage-1 output	Rows	Purpose
specifications.parquet	28,583	Base table: 174 columns of metadata, content, Markdown and recorded-history summaries.
spec_history.parquet	42,376	Recorded commits affecting specification files.
task_documents / task_links	8,253 / 22,932	Task snapshots and their associations with specifications; separate Parquets.
spec_references / spec_pr_evidence	261,032 / 89,242	Typed references and PR leads; separate Parquets.
artifact_code_refs.parquet	30,435	Collected artifact-to-code reference evidence.

All generated experiment artifacts are under results/completion_exploration/. The annotation base is deliberately richer than the later predictor table: current task contents and full-history summaries are useful for reviewing outcomes, but cannot automatically be treated as information available at an earlier prediction date.

3. Building and reviewing the annotation cohort

completion_stage2_candidates.py selected records with at least two distinct specification-file commits, available linked task text, and nonempty task lists whose parsed checkboxes were all checked. The parser handles bullet and numbered lists while ignoring examples in code fences, comments and quotations. Initially, archived specification/task paths were excluded, yielding 140 candidates. A deterministic pilot selected ten records from different repositories and task groups. Stage 3 expanded review to all 140; stage 4 admitted archived paths while retaining the other conditions, producing 564 records.

The final cohort covers 45 repositories, 491 repository/path pairs and 299 shared task groups. Every selected record is OpenSpec under the available task-content coverage. These are not 564 independent observations. The source sample is star/PR-curated; selecting fully checked lists also creates outcome-dependent selection bias. Archive membership was treated as a retrieval characteristic: archived work can have useful completion evidence, and a path move can conceal earlier history.

Cumulative review	Records	Completed	Unclear
Pilot	10	7	3
Non-archived candidates	140	69	71
Expanded cohort, archived paths included	564	305	259

Evidence protocol and a concrete implementation trace

Each record received a Markdown packet containing the selected specification, associated task snapshots, recorded spec commits and PR leads. AI-assisted review compared the specification's scope with relevant implementation, tests, documentation or maintainer evidence. The CSV records manual_status, manual_scope_match, evidence URLs, an optional supported outcome date, notes, reviewer and review date. Individual JSON decisions preserve the rationale and limitations. These judgments await human confirmation; "manual" in the field names does not establish independent human annotation.

For example, the cosmo bidirectional-checker review followed task-file history to PR #98 and commit 10db0d9f7679abf36feee3e0ee37309aac9a9708. The same commit checked off task 4.1, added mltt_check/mltt_infer in packages/cosmoc/src/types/mltt/check.cos, and supplied check_test.cos for task 4.4. Code and test definitions were compared with the required behavior; the tests were not executed. This combined direct commit linkage with reviewer judgment. It did not create an exhaustive task-by-task activity timeline.

The annotation outputs under all564/ in the experiment directory are completion_review.csv, review_population.parquet and completion_review_annotated.parquet. The annotated Parquet joins screening metadata with the saved review fields; it is not the complete source table or the prediction-time feature matrix. completion_apply_annotations.py applies saved decisions without inferring labels; completion_summarize_review.py exports the annotated Parquet and counts.

The 305 completion judgments include 86 with supported evidence dates and 219 without dates. Unclear means evidence does not establish the selected completion scope; it must not become "abandoned," "unfinished," or a negative training label. Checked tasks can coexist with deferrals or scope discrepancies, while an unchanged specification can already be mature and successfully implemented.

4. Preparing data available at a prediction cutoff

completion_stage5_cutoff.py preserves the 564 identifiers and full-history manual_status values, but recomputes predictors using only specification commits at or before the cutoff. For a window W, prediction_at equals the earliest recorded spec-path commit plus W × 24 hours, in UTC. This anchor is not verified creation: 23 dated completions precede it, showing why archive/rename lineage matters. The label may use later evidence; the predictors may not.

The 12 predictors summarize commit count, active calendar days, distinct author names, missing author names, activity span, inactivity at cutoff, mean/maximum gaps, first-/second-half counts, and trailing three-/seven-day counts. They measure specification editing, not all work on the associated tasks. Current Markdown metrics, checked-task counts, repository snapshots and latest PR states are excluded because their historical values were not reconstructed. Only feature_contract.json's explicit predictor list belongs in a later model's inputs.

window_observed compares the cutoff with commit_history_fetched_at. It means collection covered the requested period, not that the spec remained active. If collection ended earlier, the record and intended cutoff remain, but all 12 predictors are null. The output also retains the bounded commit history for inspection. Seven days is an exploratory coverage default; 90 days retains almost all 1,309 recorded spec/commit pairs, including shared events across records.

Days	Observed windows	Retained commits	Completed before cutoff	Baseline-review queue
7	561	814	77	486
14	560	898	82	480
30	552	1,028	86	473
60	410	1,249	86	361
90	368	1,304	86	332

Stage 6 adds status_at_cutoff, nullable completed_by_cutoff, separate has_recent_spec_commit, evidence/reason fields and training_row_action. Prior completion requires the reviewed evidence date to precede the cutoff's calendar day; same-day, later and undated evidence leave status unknown. There is no inferred active/inactive flag. The master retains every row. Known-completed cases are excluded from forecasting first completion; other incomplete windows are excluded, and remaining records enter a review queue. Completion takes priority when both exclusions apply. No queue is automatically a training set.

The latest tables are cutoff_status/day_007/cutoff_dataset.parquet and corresponding day_014, day_030, day_060 and day_090 files, each with 564 rows and 26 columns. CSV companions support inspection; needs_baseline_review files contain the queues. All are relative to the experiment directory. Their new status/evidence fields are metadata, excluded from predictors.

The model remains untrained: no confirmed negative class, verified in-progress baseline cohort, agreed outcome horizon, accuracy, precision or recall exists. Future evaluation should separate repositories and related records. Unfinished required tasks plus sustained absence of related code/task/PR activity is a candidate abandonment signal to validate, not an established label rule. That work requires broader sampling and historical task evidence; the present cohort was selected using fully checked lists.

5. Reproducing the experiment after committing the code

Commit the completion scripts, tests, dependency files, .gitattributes, report sources and reproducibility/completion_annotations/. This bundle preserves 564 decision JSONs, original cohort subsets and hashes. Code alone cannot recreate evidence judgments: replay restores the saved review, without rerunning AI annotation. Back up public source caches separately for later evidence auditing. Large raw Parquets come from the pinned SpecMine checkout.

Clean checkout: commands from the repository root

Use Python 3.13 (the experiment used 3.13.2). Create an environment and restore the exact data snapshot. Skip cloning only when that pinned source is already present:

```
py -3.13 -m venv .venv-completion
$py = '.\.venv-completion\Scripts\python.exe'
& $py -m pip install -r reproducibility/requirements-completion.txt
git clone https://github.com/shyamagarwal13/specmine-official.git `
  data/raw/specmine-official
git -C data/raw/specmine-official checkout `
  9fbb61541046cb2fb770fa944192a3fc0b64a9b1
& $py src/completion_reproduce.py
& $py -m unittest discover -s tests -p 'test_completion*.py' -q
```

completion_reproduce.py validates inputs and runs the stages into results/completion_reproduction/. For another run, supply a fresh destination with --output-root results/completion_reproduction_2. The wrapper saves commands.json, reproduction.log and reproduction_verification.json. Individual scripts remain runnable; all names below are under src/:

Script(s)	Role in execution order
completion_stage1_prepare.py	Consolidate the base and evidence Parquets.
completion_stage2_candidates.py	Screen 140 candidates and prepare pilot ten.
completion_stage3_review_all.py completion_stage4_include_archived.py	Expand to 140, then 564; carry prior judgments.
completion_apply_annotations.py completion_summarize_review.py	Apply saved decisions at each review stage; export labels and summaries.
completion_stage5_cutoff.py completion_stage6_status.py	Build five cutoff versions, then supported status flags and queues.
completion_verify_expansion.py completion_verify_cutoff.py	Verify population, labels, temporal filtering, features and provenance.

Validation and interpretation

All 54 completion tests passed, covering joins, checklist parsing, annotation preservation, cutoff boundaries, missing observations and independence from future commits. The clean replay passed all 13 pipeline commands: the 10-, 140- and 564-record review CSVs matched the originals byte for byte, and cutoff/status summaries matched. Independent artifact verifiers also passed. These establish reproducibility and implementation consistency, not label reliability or predictive performance.

Detailed methods and commands remain in ROUND_2_LOG.md, COMPLETION_EXPERIMENT.md, COMPLETION_ALL564_REVIEW.md, COMPLETION_CUTOFF_EXPERIMENT.md and CUTOFF_STATUS_EXPERIMENT.md. The central result is a reproducible evidence and feature workflow, with explicit records of what must be resolved before training a model that forecasts unfinished work's outcome.